

Chapter 8

Deadlocks

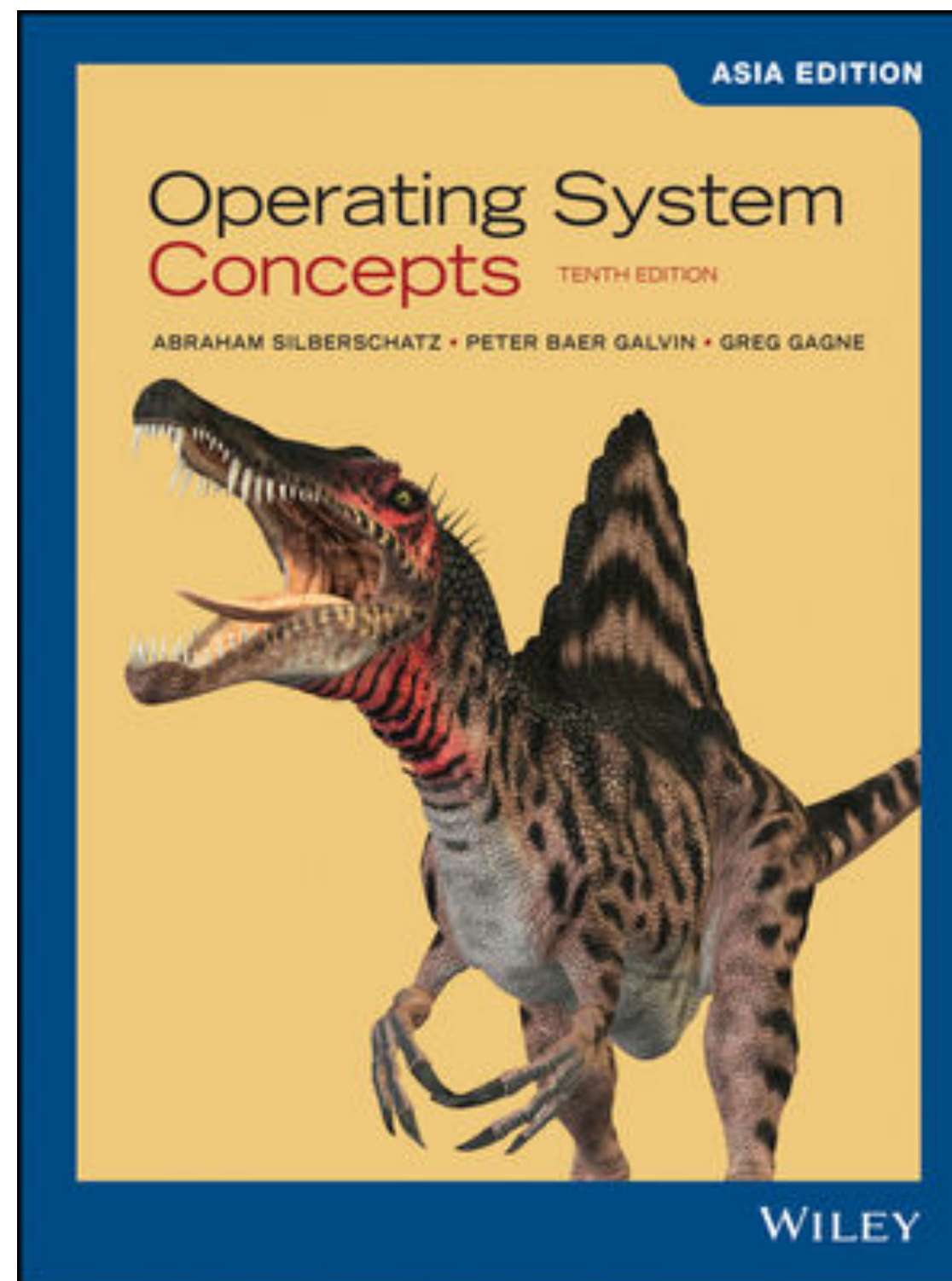
Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/5/13

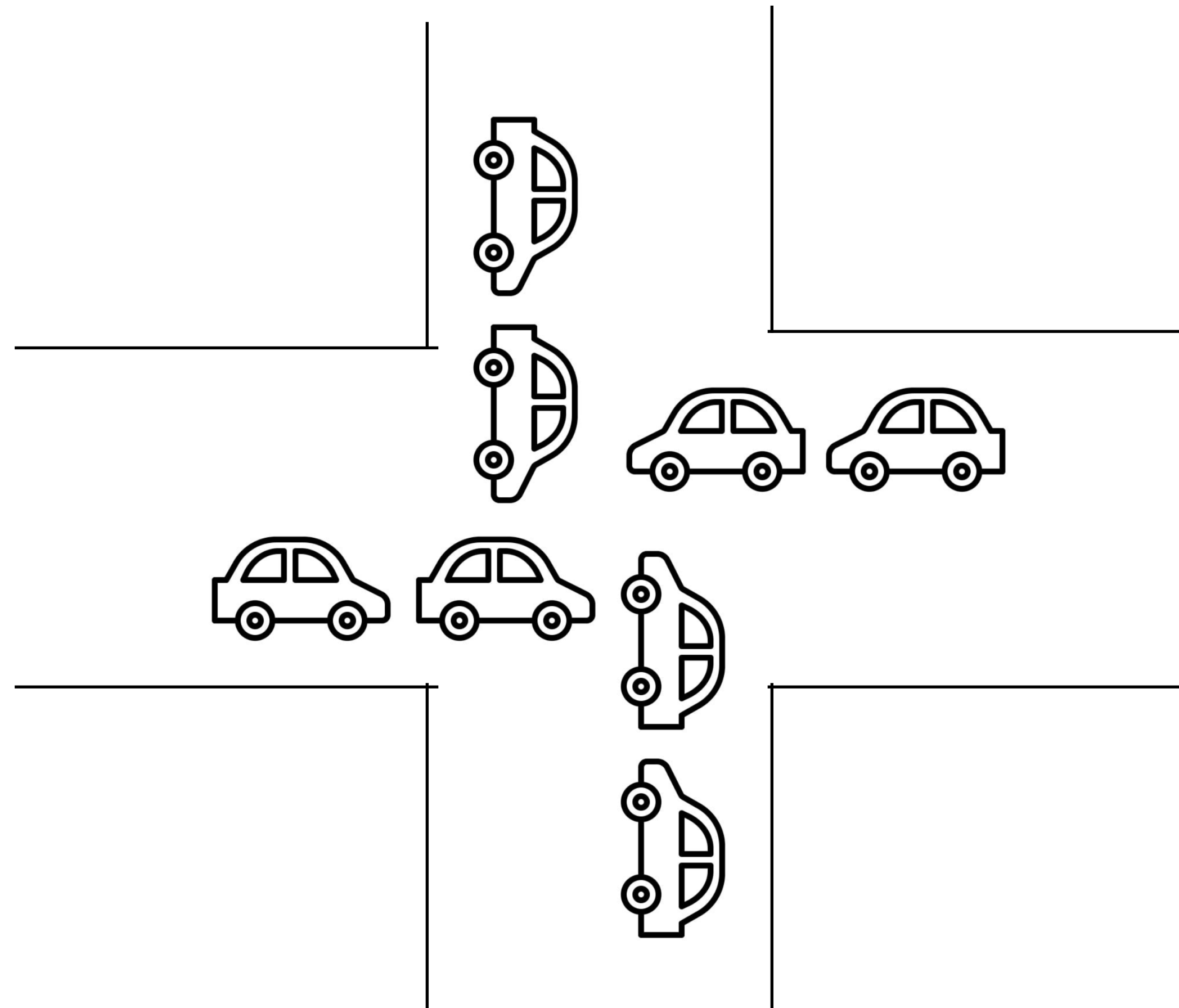
Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Overview

- Every process in a set of processes is waiting for an event that can be caused only by another process in the set



Deadlock example (Java)

(tronclass 上有 C++ 版本)

```
ReentrantLock first_mutex = new ReentrantLock();
ReentrantLock second_mutex = new ReentrantLock();

void do_work_one() {

    first_mutex.lock();

    System.out.println("do_work_one: first_mutex.lock()");

    try {
        Thread.sleep(1000);
    } catch (Exception e) {
    }

    second_mutex.lock(); → 要等 do-work-two
                        所以要不到 second 的 mutex lock

    System.out.println("do_work_one");

    second_mutex.unlock();
    first_mutex.unlock();
}
```

Deadlock example (Java)

Easy solution
(for this program):
do_work_two 的
first_mutex.lock() &
second_mutex.lock() 对调!
→ 避免循环等待

```
void do_work_two() {  
    second_mutex.lock();  
  
    System.out.println("do_work_two: second_mutex.lock()");  
  
    first_mutex.lock();  
  
    System.out.println("do_work_two");  
  
    first_mutex.unlock();  
    second_mutex.unlock();  
}
```

→ 要等 do_work_one
要不到 first 的 mutex lock

Outline

- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

Outline

- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

Deadlock Characterization

必考

- Necessary Conditions → 是必要條件還是充要條件取決於系統設計
 - $P(\text{Deadlock}) \Rightarrow Q(\text{Four conditions})$
 - P cannot be true unless Q is true
 - <https://www.thenewslens.com/article/139175>
- Four conditions hold simultaneously 發生 Deadlock 時, 必有 4 條件必 同時 發生 (所以, Deadlock 其實不容易發生)
 - Mutual exclusion 條件 1: 互斥 (想建立 Critical Section)
 - Hold and wait 條件 2: 持有一個資源 且 同時等待另一個資源
 - No preemption (強取豪奪) 條件 3: 非搶佔式 (一個 thread 不能搶其他 thread 的資源)
 - Circular wait 條件 4: 循環等待 (每一個 thread 都在等另一個 thread)

Four Conditions

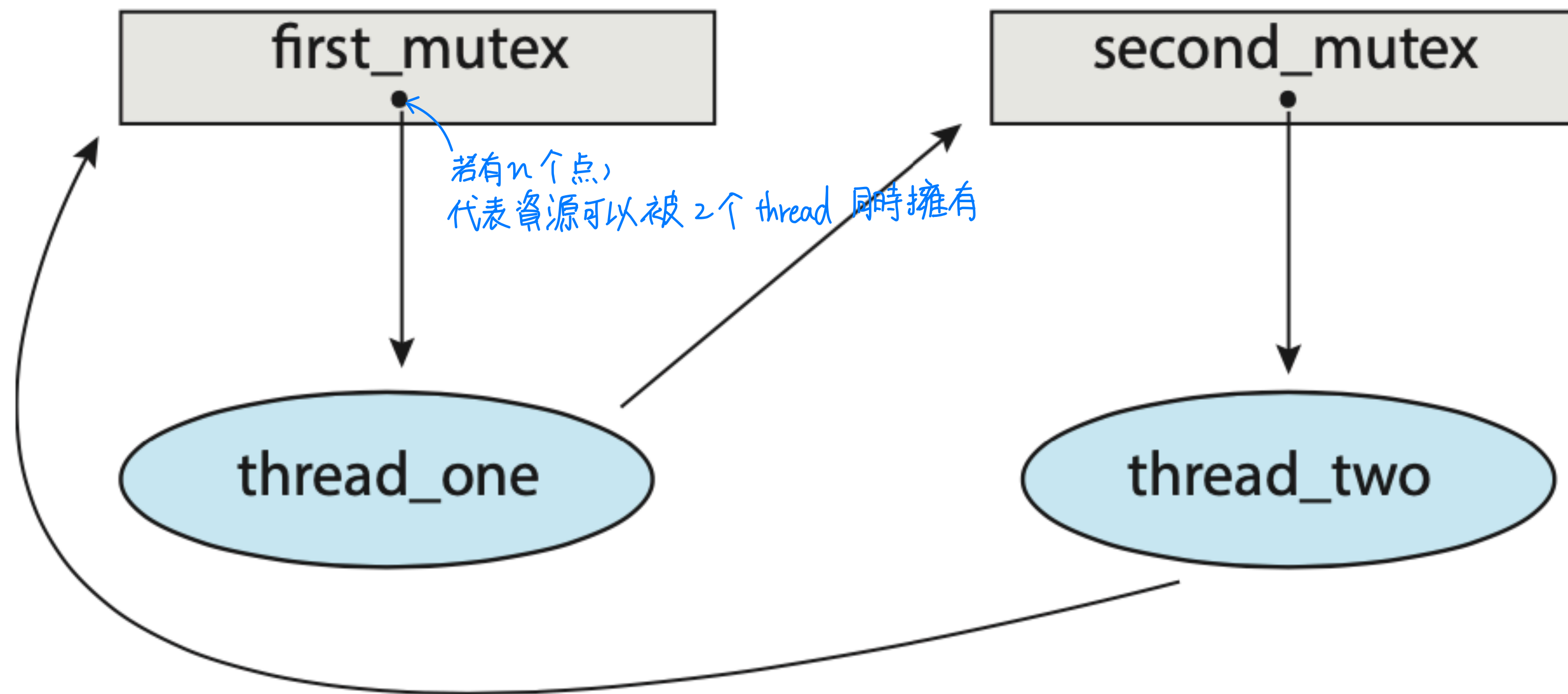
- Mutual Exclusion
 - At least one resource must be held in a non-sharable mode
- Hold and wait
 - A thread must be holding at least one resource and waiting to acquire additional resources that are currently being held by other threads
- No preemption
 - Resources cannot be preempted
- Circular wait
 - $T_1 > T_2 > T_3 > \dots > T_n > T_1$

Resource-Allocation Graph

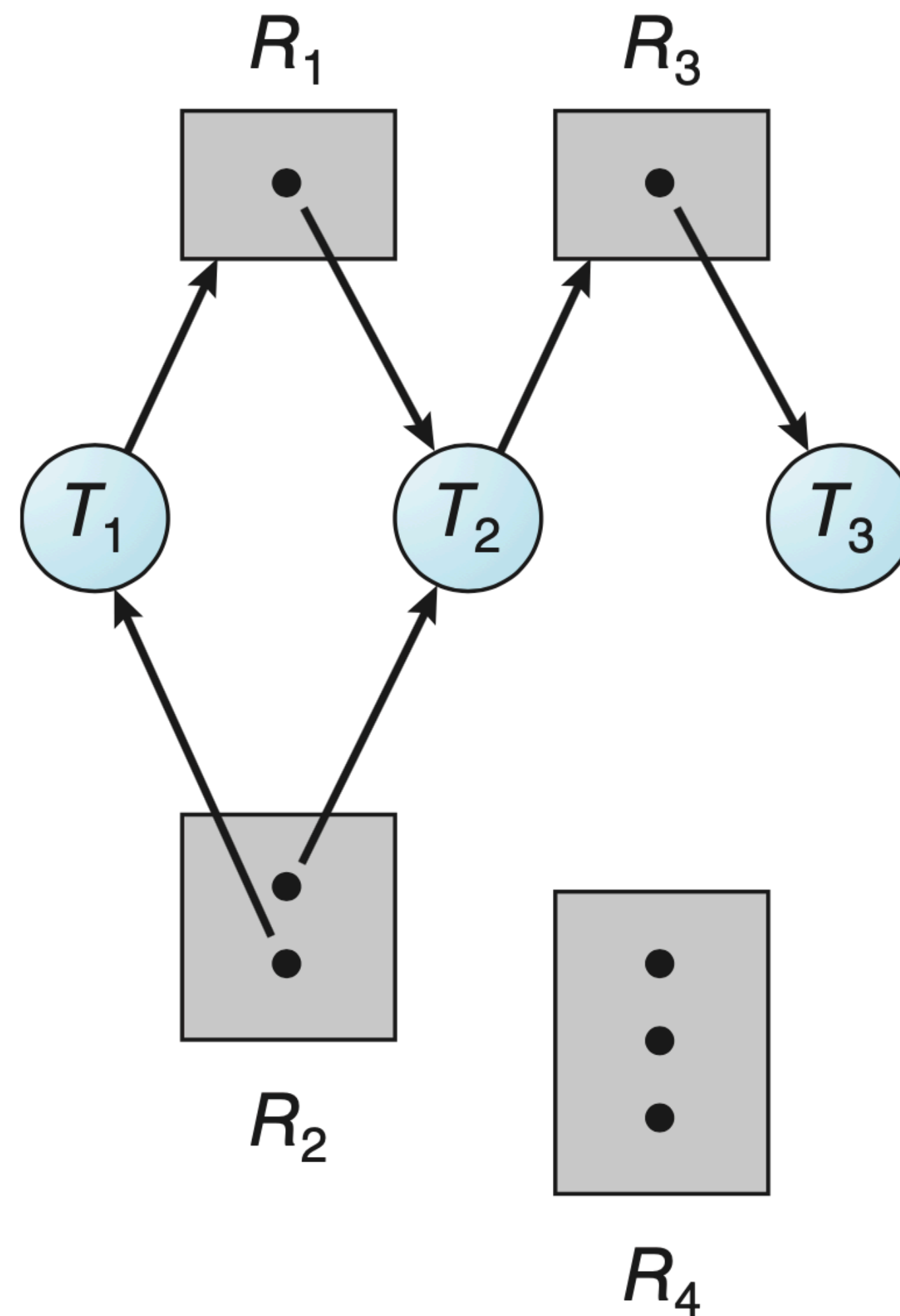
- A set of vertices ^{頂点} V
 - Two different types of nodes
 - Threads $T = \{T_1, T_2, \dots, T_n\}$
 - Resources $R = \{R_1, R_2, \dots, R_m\}$
- A set of directed edges ^辺 E
 - A directed edge from T_i to R_j (**request edge**): T_i has requested an instance of R_j and is currently waiting for that resource
 - A directed edge from R_j to T_i (**assignment edge**): an instance of R_j has been allocated to T_i
 - Instances of R_j : we represent each instance as a dot within the rectangle

Example (Deadlock Situation)

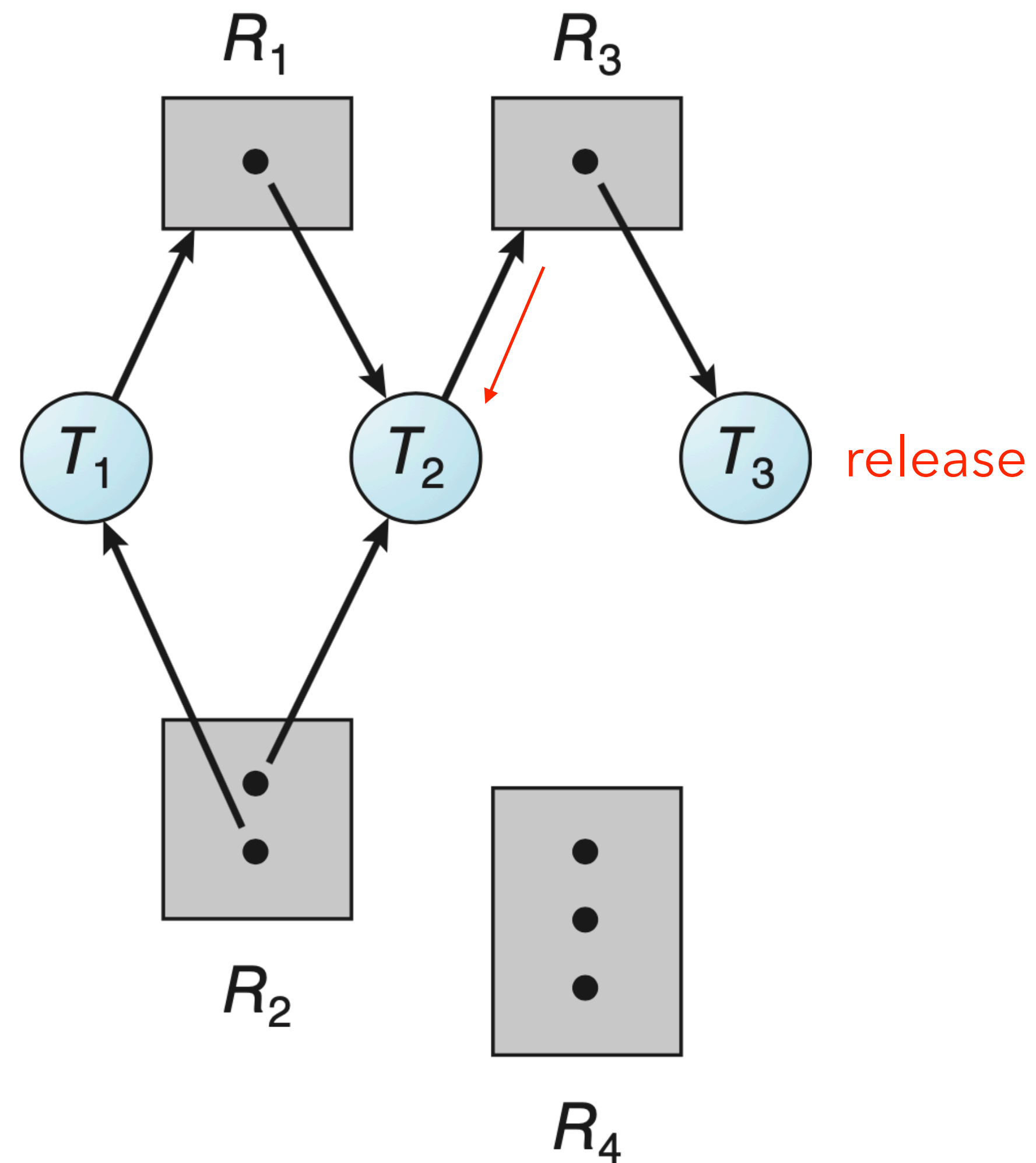
若图能够形成一个 Cycle, 代表发生 Dead lock



Example



Example



Resource-Allocation Graph

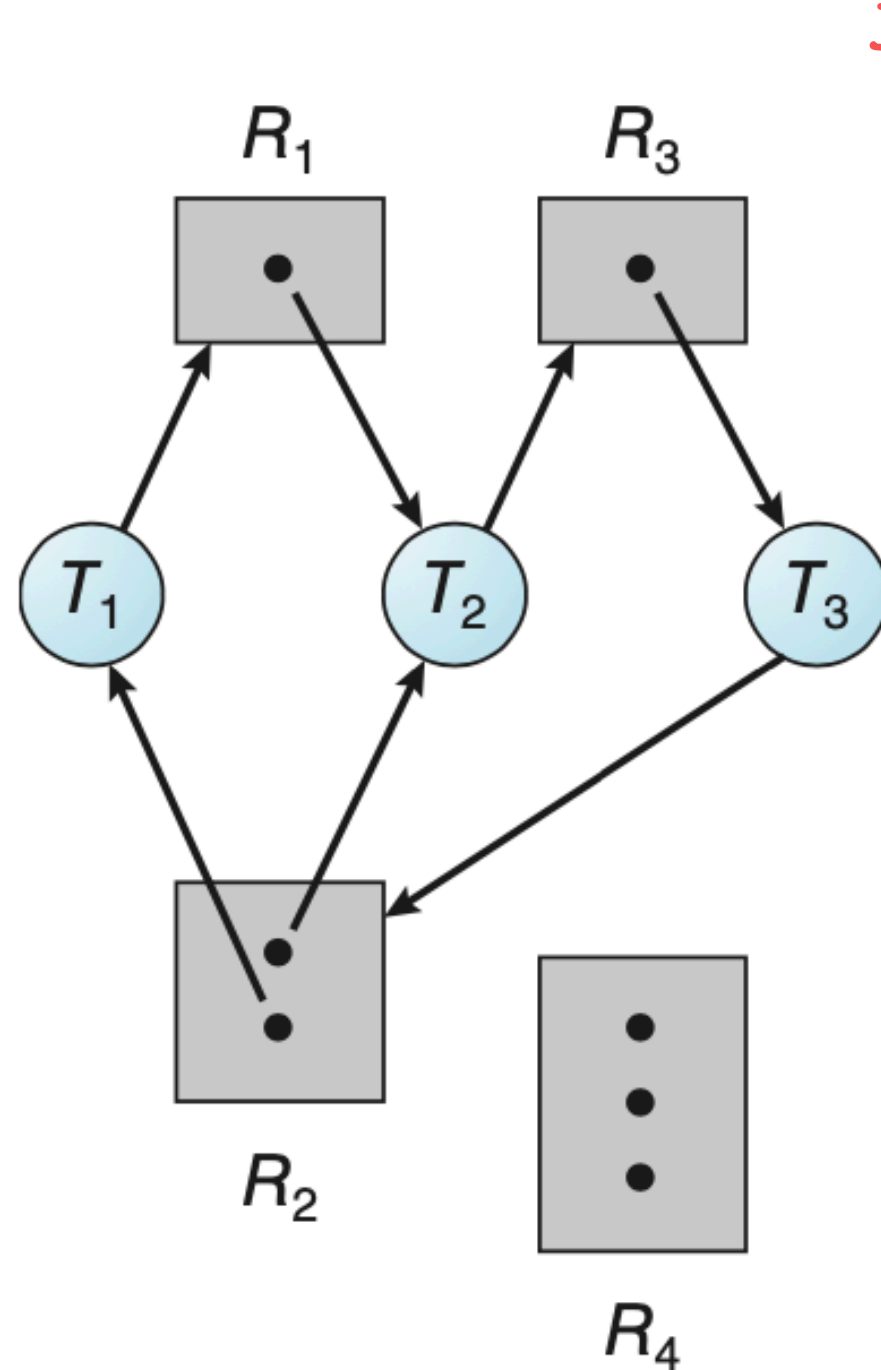
- If the graph contains no cycles, then no thread in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist.
Cycle 是 Deadlock 的必要條件, 不是充分條件
- One instance
 - If each resource has exactly one instance, then a cycle implies that a deadlock has occurred.
 - If the cycles involves only a set of resources types, each of which has only a single instance, then a deadlock has occurred.
 - A cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.

Resource-Allocation Graph

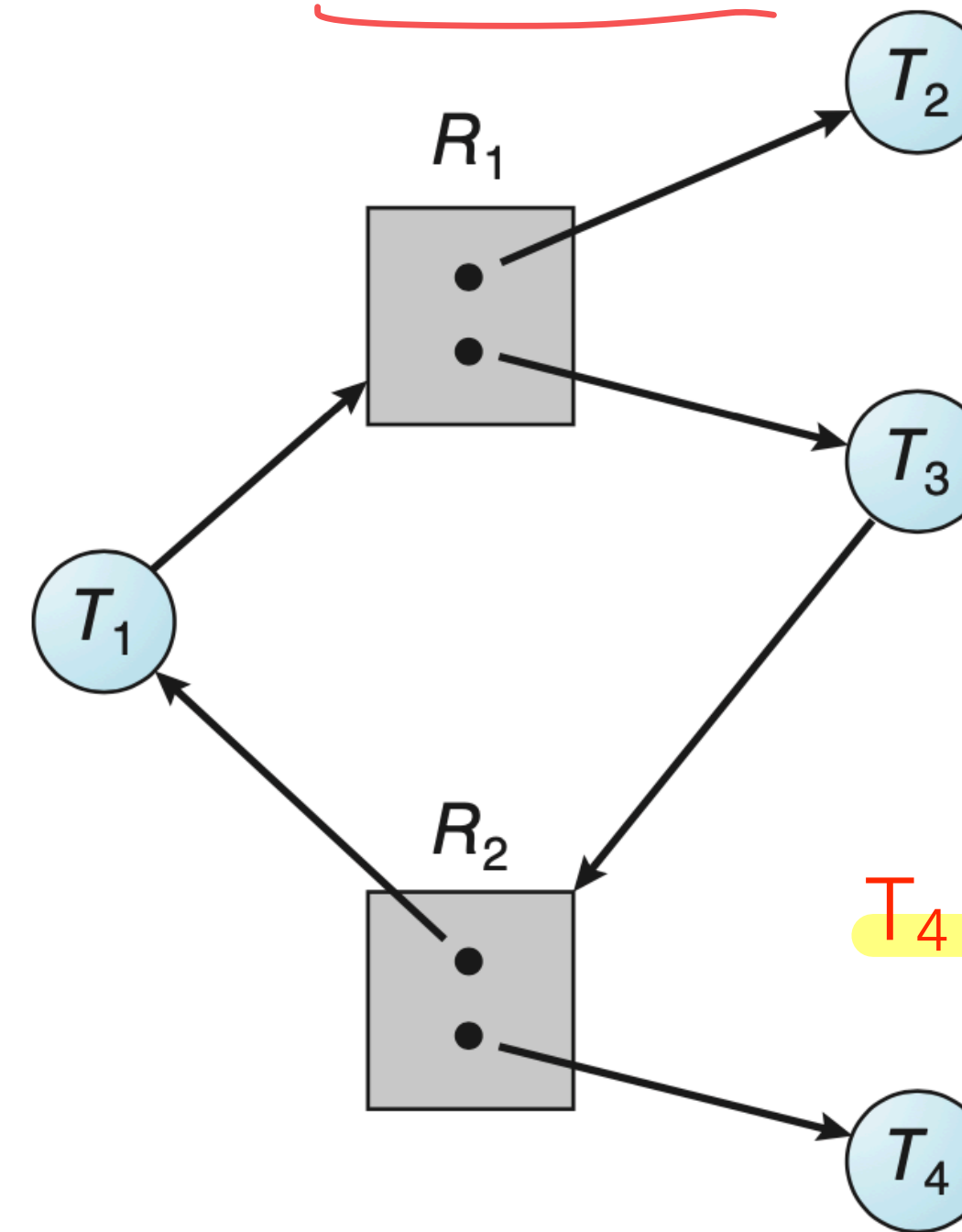
- Multiple instances

必要條件但非充分條件

- A cycle in the graph is a necessary but not a sufficient condition.



Resource-Allocation Graph with a deadlock



T_4 may release its instance of R_2

Resource-Allocation Graph with a cycle but no deadlock

Summary

- If a resource-allocation graphs does not have a cycle, the system is **not** in a deadlocked state
- If there is a cycle, then the system **may** or **may not** be in a deadlocked state

one instance 時，有 cycle 就一定 Dead Lock

Outline

- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention for programmer to prevention.
- Deadlock Avoidance for OS (legacy)
- Deadlock Detection for OS (legacy)
- Recovery from Deadlock

→ 核心思想：不讓 DL 發生

→ 讓 DL 發生，想辦法偵測並解決他

現在作業系統不處理 DL，當作沒發生！

會考
不考

Methods for Handling Deadlocks

- We can ignore the problem altogether and pretend that deadlocks never occur in the system. *現代的 OS 是不處理 Dead Lock 的, 當作 Deadlock 沒發生 (因為 Time Sharing)
 - Linux / Windows
- We can use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.
 - Programmers
 - Deadlock Prevention (8.5) / Deadlock Avoidance (8.6)
- We can allow the system to enter a deadlocked state, detect it, and recover.
 - Some database systems
 - Deadlock Detection (8.7) / Recovery from Deadlock (8.8)

Outline

- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

Deadlock Prevention

- By ensuring that at least one of four conditions cannot hold, we can prevent the occurrence of a deadlock.

Deadlock Prevention

- Mutual Exclusion

- We cannot prevent deadlocks by denying the mutual-exclusion condition, because some resources are intrinsically non-sharable.
- e.g., a mutex lock

→ 確保 Mutual Exclusion 不發生 → 不建立任何 C.S., 不使用任何 SpinLock、Semaphores → race conditions.

Deadlock Prevention

- Hold and Wait

- Require each thread to request and be allocated all its resource before it begins execution.
- Allow a thread to request resources only when it has none. Before it can request any additional resources, it must release all the resources that it is currently allocated.
- These two protocols have two disadvantages
 - Resource utilization may be low
 - Starvation is possible

→ 整個程式碼只使用一個

SpinLock / Semaphores

→ 資源利用沒效率！

Deadlock Prevention

- No Preemption 禁止搶奪

- If a thread is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources the thread is currently holding are preempted.
- If a thread requests some resources that are not available, we check whether they are allocated to some other thread that is waiting for additional resources. If so, we preempt the desired resources from the waiting thread and allocate them to the requesting thread.

→ 允許強奪就好了 → System Call 去 kill 其他 process

Deadlock Prevention

4種避免方法中
最好實作、理解的

• Circular Wait

- One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each thread requests resources in an increasing order of enumeration.
- We assign to each resource type a **unique** integer number
 - e.g., $F(\text{first_mutex}) = 1$, $F(\text{second_mutex}) = 5$
- A thread can initially request an instance of a resource (R_i)
- After that, the thread can request an instance of resource R_j if and only if $F(R_j) > F(R_i)$

SpinLock / Semaphores

如果已經要過編號3

不能回頭去要2跟1

Programmer 怎麼
避免 Circular Wait?

→ 將 Spin Lock
都編號

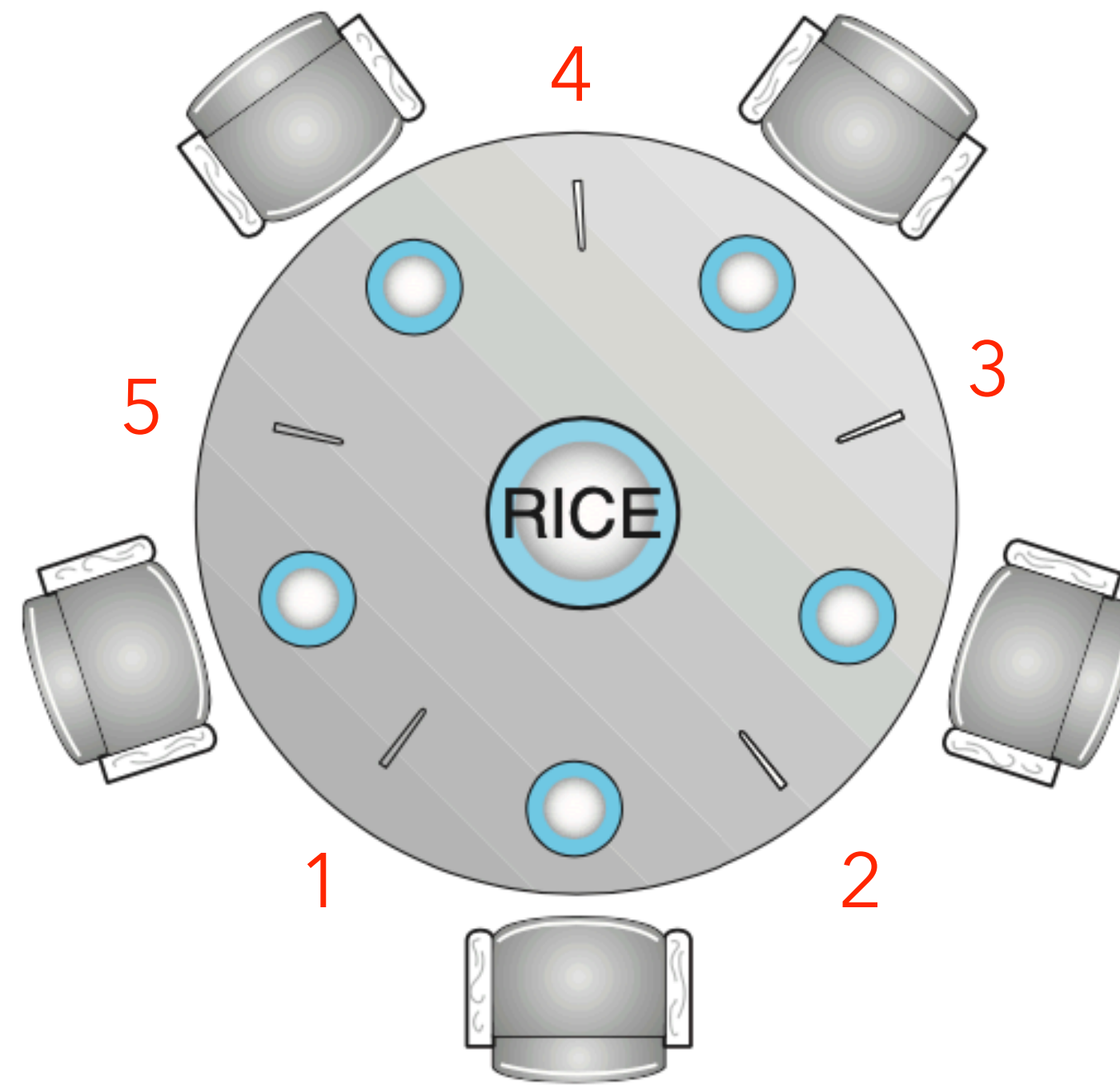
呼叫 Lock 時

→ 都必須按照順序!

(3 5 1 . 9 2 3 x)
(1 3 5 . 2 3 9 ✓)

Deadlock Prevention

- Circular Wait



Deadlock Prevention

```
void do_work_one() {
```

```
    first_mutex.lock();  
    second_mutex.lock();
```

```
    System.out.println("do_work_one");
```

```
    second_mutex.unlock();  
    first_mutex.unlock();
```

```
}
```

```
void do_work_two() {
```

```
    first_mutex.lock();  
    second_mutex.lock();
```

```
    System.out.println("do_work_two");
```

```
    first_mutex.unlock();  
    second_mutex.unlock();
```

```
}
```


Outline

- Deadlock Characterization
 - Methods for Handling Deadlocks
 - Deadlock Prevention
 - Deadlock Avoidance
 - Deadlock Detection
 - Recovery from Deadlock
- 沒考過！

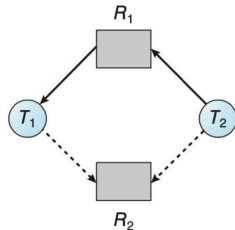
Deadlock Avoidance

要求每个 Process / Thread 執行前都先通知作業系統要多少資源

- To require additional information about how resources are to be requested
- Example
 - A system with resources R_1 and R_2
 - Thread P will request first R_1 and then R_2 before releasing both resources
 - Thread Q will request R_2 , and then R_1
 - With this knowledge, the system can decide for each request whether or not the thread should wait in order to avoid a possible future deadlock

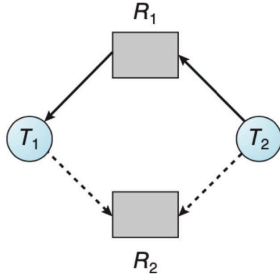
Resource-Allocation-Graph Algorithm

- Assume a system with only one instance of each resource type
- A set of directed edges E
 - Request edge
 - Assignment edge
 - Claim edge
- A claim edges from T_i to R_j indicates that thread T_i may request resource R_j at some time.
- The resources must be claimed a priori in the system
作業系統會幫忙判斷 Process / Thread 是否能拿到 / 釋放資源

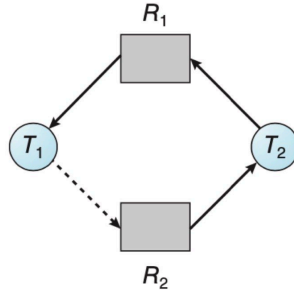


Resource-Allocation-Graph Algorithm

- Suppose that thread T_i requests resource R_j . The request can be granted only if covering the request edge $T_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow T_i$ does not result in the formation of a cycle.



T_2 requests R_2



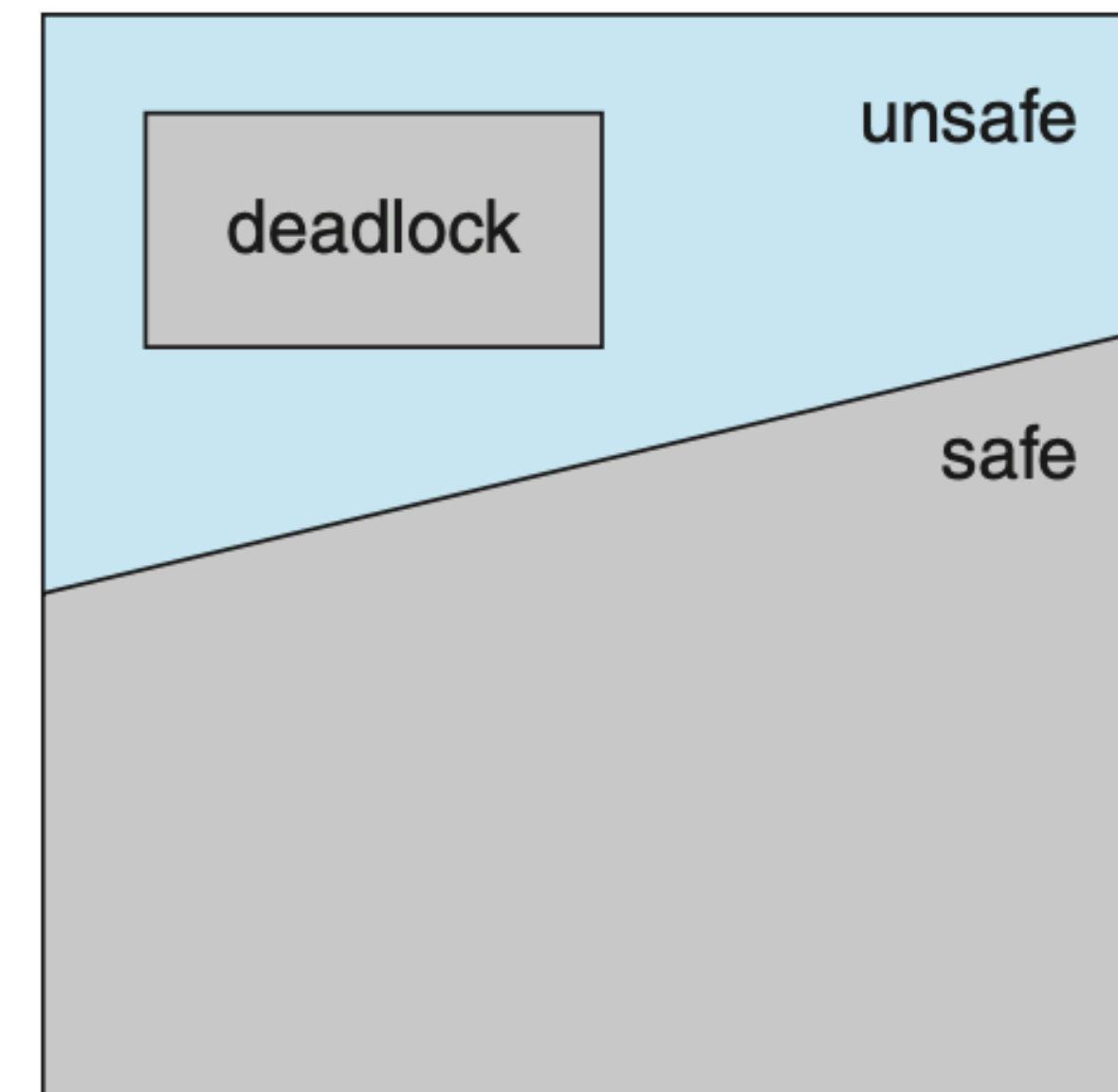
An unsafe state in a resource-allocation graph.

Deadlock Avoidance

- The various algorithms that use this approach (deadlock avoidance) differ in the amount and type of information required
- The simplest and most useful model requires that each thread **declare** the **maximum number** of resources of each type that it may need
- Deadlock-avoidance algorithms
 - Resource-Allocation-Graph Algorithm
 - Banker's Algorithm

Safe State

- A state is **safe** if the system can allocate resource to each thread (up to its maximum) in some order and still avoid a deadlock.
- More formally, a system is in a **safe** state only if there exists a **safe sequence**.
- A sequence of threads $\langle T_1, T_2, \dots, T_n \rangle$ is a **safe sequence** for the current allocation state if for each T_i , the resource requests that T_i can still make can be satisfied by the currently available resources plus the resource held by all T_j , where $j < i$.
- If no such sequence exists, then the system state is said to be **unsafe**.



Example

- Consider a system with 12 resources, and 3 threads.

	Maximum Needs	Current Needs
T₀	10	5
T₁	4	2
T₂	9	2

这些资源都要
提前跟 OS 说

Example

- The sequence $\langle T_1, T_0, T_2 \rangle$ satisfies the safety condition
 - $T_1: 3 \text{ (available)} + (2) \geq 4$
 - $T_0: 3 \text{ (available)} + 2 + (5) \geq 10$
 - $T_2: 3 \text{ (available)} + 2 + 5 + (2) \geq 9$
- The sequence $\langle T_0, T_1, T_2 \rangle$
 - $T_0: 3 \text{ (available)} + (5) < 10$

Example

- Consider a system with 12 resources, and 3 threads.

	Maximum Needs	Current Needs
T₀	10	5
T₁	4	2
T₂	9	3

Example

- The sequence $\langle T_1, T_0, T_2 \rangle$ does not satisfy the safety condition
 - T_1 : $2 \text{ (available)} + (2) \geq 4$
 - T_0 : $2 \text{ (available)} + 2 + (5) < 10$
 - T_2 : $2 \text{ (available)} + 2 + 5 + (3) \geq 9$
- Our mistake was in granting the request from thread T_2 for one more resource
- Whenever a thread requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or the thread must wait

Banker's Algorithm

不考

- A system with multiple instances of each resource type.

Banker's Algorithm

- Data Structures (1/2)
 - **Available**
 - A **vector** of length m indicates the number of available resources of each type
 - If **Available**[j] equals k , then k instances of resource type R_j are available
 - **Max**
 - An $n \times m$ **matrix** defines the maximum demand of each thread
 - If **Max**[i][j] equals k , then thread T_i may request at most k instances of resource type R_j

Banker's Algorithm

- Data Structures (2/2)
 - **Allocation**
 - An $n \times m$ **matrix** defines the number of resources of each type currently allocated to each thread
 - If ***Allocation*** $[i][j]$ equals k , then thread T_i is currently allocated k instances of resource type R_j
 - **Need**
 - An $n \times m$ **matrix** indicates the remaining resource need to each thread
 - ***Need*** $[i][j] = \mathbf{Max}[i][j] - \mathbf{Allocation}[i][j]$

Banker's Algorithm

- Notation
 - Let X and Y be vectors of length n . We say that $X \leq Y$ if and only if $X[i] \leq Y[i]$ for all $i = 1, 2, \dots, n$.
 - The vector ***Allocation*** _{i} specifies the resources currently allocated to thread T_i
 - The vector ***Need*** _{i} specifies the additional resources that thread T_i may still request to complete its task

Safety Algorithm

- This algorithm is used for finding out whether or not a system is in a safe state
 1. Let *Work* and *Finish* be vectors of length m and n , respectively. Initialize *Work* = *Available* and *Finish*[i] = *false* for $i = 0, 1, \dots, n - 1$.
 2. Find an index i such that both
 - a. *Finish*[i] == *false*
 - b. $Need_i \leq Work$If no such i exists, go to step 4.
 3. *Work* = *Work* + *Allocation* _{i}
Finish[i] = *true*
Go to step 2.
 4. If *Finish*[i] == *true* for all i , then the system is in a safe state.

Resource-Request Algorithm

- This algorithm is used for determine whether requests can be safely granted
- Let $Request_i$ be the request vector for thread T_i . If $Request_i[j] = k$, then thread T_i wants k instances of resource type R_j .

1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise an error condition, since the thread has exceeded its maximum claim.
2. If $Request_i \leq Available$, go to step 3. Otherwise, T_i must wait, since the resources are not available.
3. Have the system pretend to have allocated the requested resources to thread T_i by modifying the state as follows:

$$\begin{aligned} Available &= Available - Request_i \\ Allocation_i &= Allocation_i + Request_i \\ Need_i &= Need_i - Request_i \end{aligned}$$

If the resulting resource-allocation state is safe, the transaction is completed, and thread T_i is allocated its resources. However, if the new state is unsafe, then T_i must wait for $Request_i$, and the old resource-allocation state is restored.

Example

- Five Threads: T_0 through T_4
- Three resource types A, B, and C
 - A: 10 instances
 - B: 5 instances
 - C: 7 instances

Example

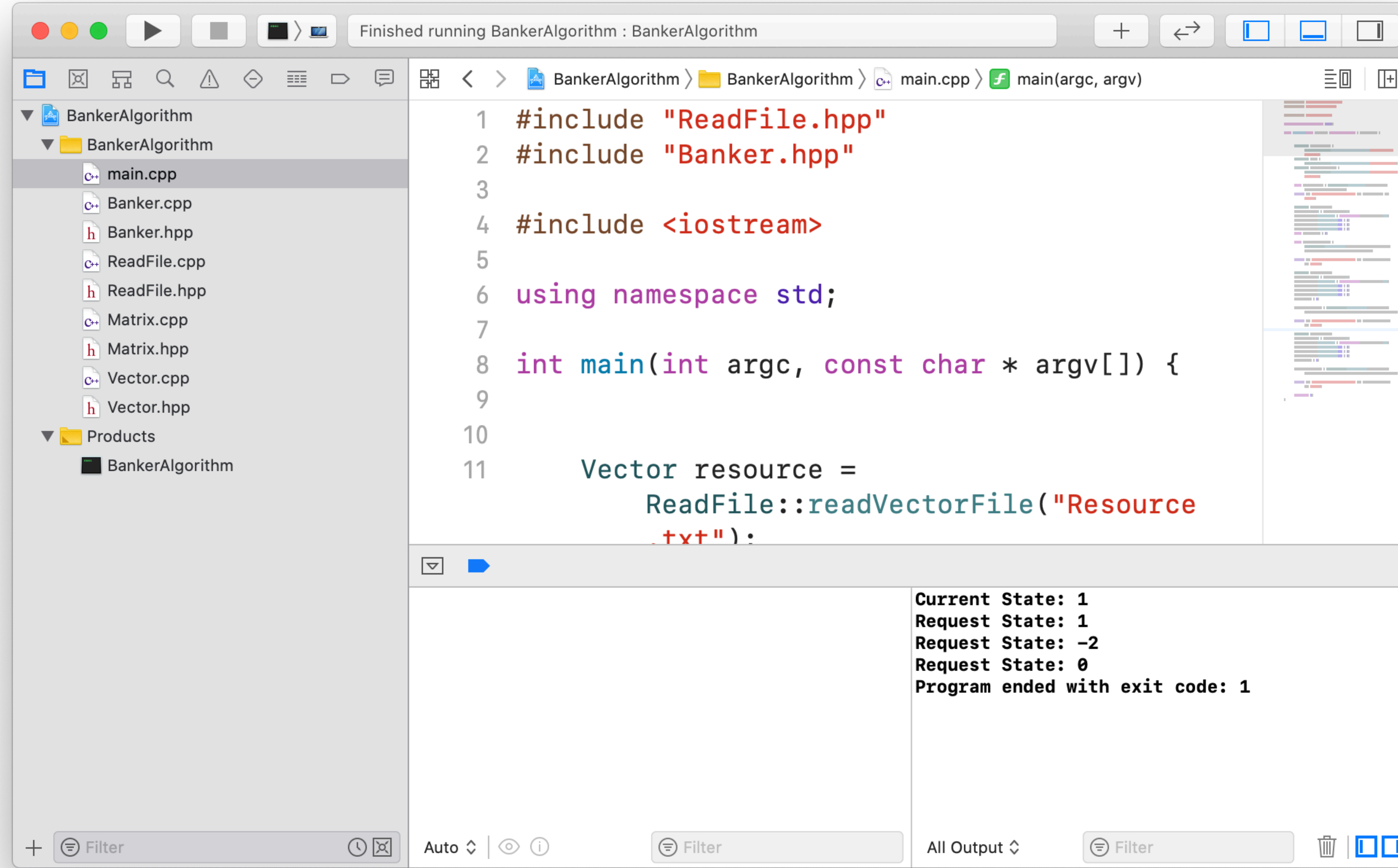
- Current State
 - The system is currently in a safe state

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>	<u>Need</u>
	A B C	A B C	A B C	A B C
T_0	0 1 0	7 5 3	3 3 2	7 4 3
T_1	2 0 0	3 2 2		1 2 2
T_2	3 0 2	9 0 2		6 0 0
T_3	2 1 1	2 2 2		0 1 1
T_4	0 0 2	4 3 3		4 3 1

Example

- Thread 1: Request₁ = (1, 0, 2)
 - Safe
- Thread 4: Request₄ = (3, 3, 0)
 - The resources are not available
- Thread 0: Request₀ = (0, 2, 0)
 - Cannot be granted, even though, the resources are available, since the resulting state is unsafe

Implementation



Outline

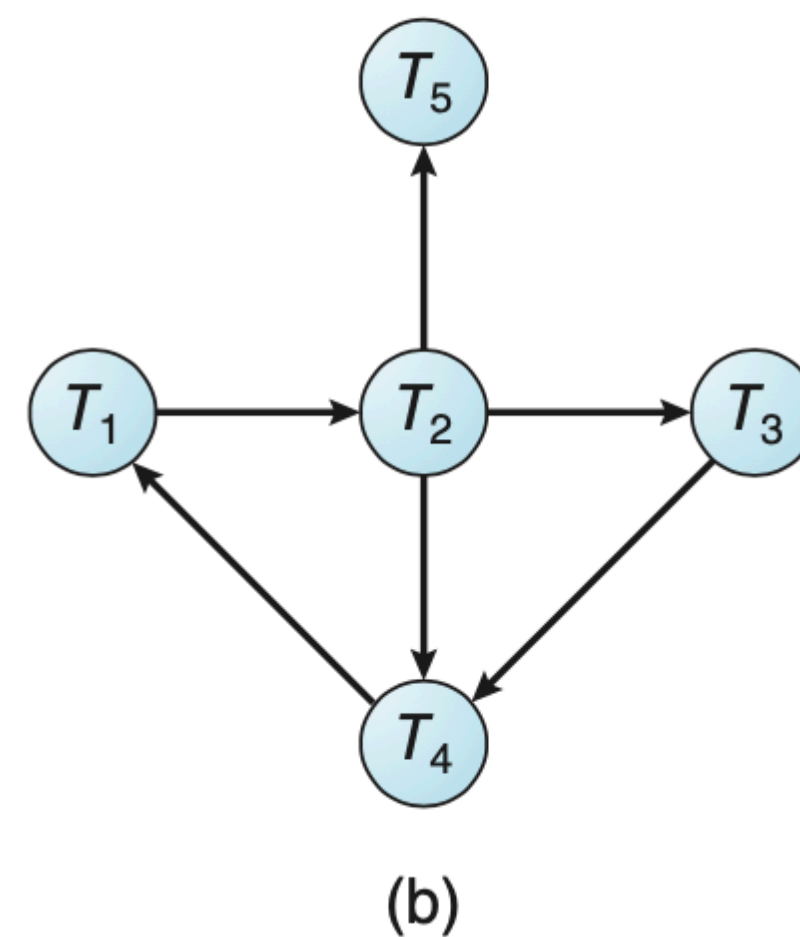
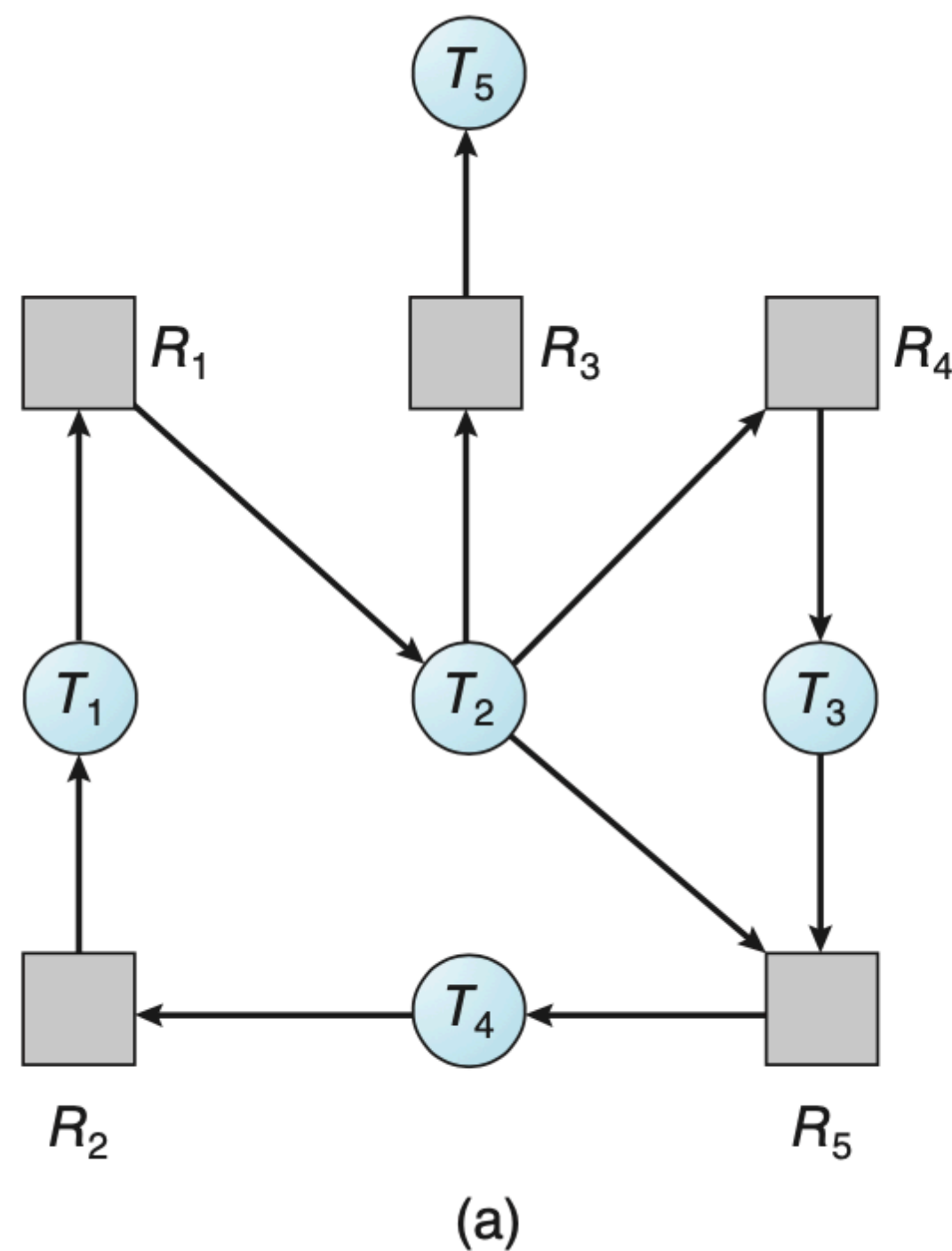
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

Deadlock Detection

- A system that does not imply either a deadlock-prevention or a deadlock-avoidance algorithm may provide
 - An algorithm can determine whether a deadlock has occurred (**Detection**)
 - An algorithm to recover (**Recovery**) from the deadlock

Single Instance of Each Resource Type

- A **wait-for** graph obtained from the resource-allocation graph by removing the resource nodes and collapsing the appropriate edges
- An edge $T_i \rightarrow T_j$ exists in a wait-for graph if and only if the corresponding resource-allocation graph contains two edges $T_i \rightarrow R_q$ and $R_q \rightarrow T_j$ for some resource R_q .



A deadlock exists in the system if and only if the wait-for graph contains a cycle.

Several Instances of a Resource Type

1. Let *Work* and *Finish* be vectors of length m and n , respectively. Initialize *Work* = *Available*. For $i = 0, 1, \dots, n-1$, if $Allocation_i \neq 0$, then $Finish[i] = false$. Otherwise, $Finish[i] = true$.
2. Find an index i such that both
 - a. $Finish[i] == false$
 - b. $Request_i \leq Work$If no such i exists, go to step 4.
3. $Work = Work + Allocation_i$
 $Finish[i] = true$
Go to step 2.
4. If $Finish[i] == false$ for some $i, 0 \leq i < n$, then the system is in a deadlocked state. Moreover, if $Finish[i] == false$, then thread T_i is deadlocked.

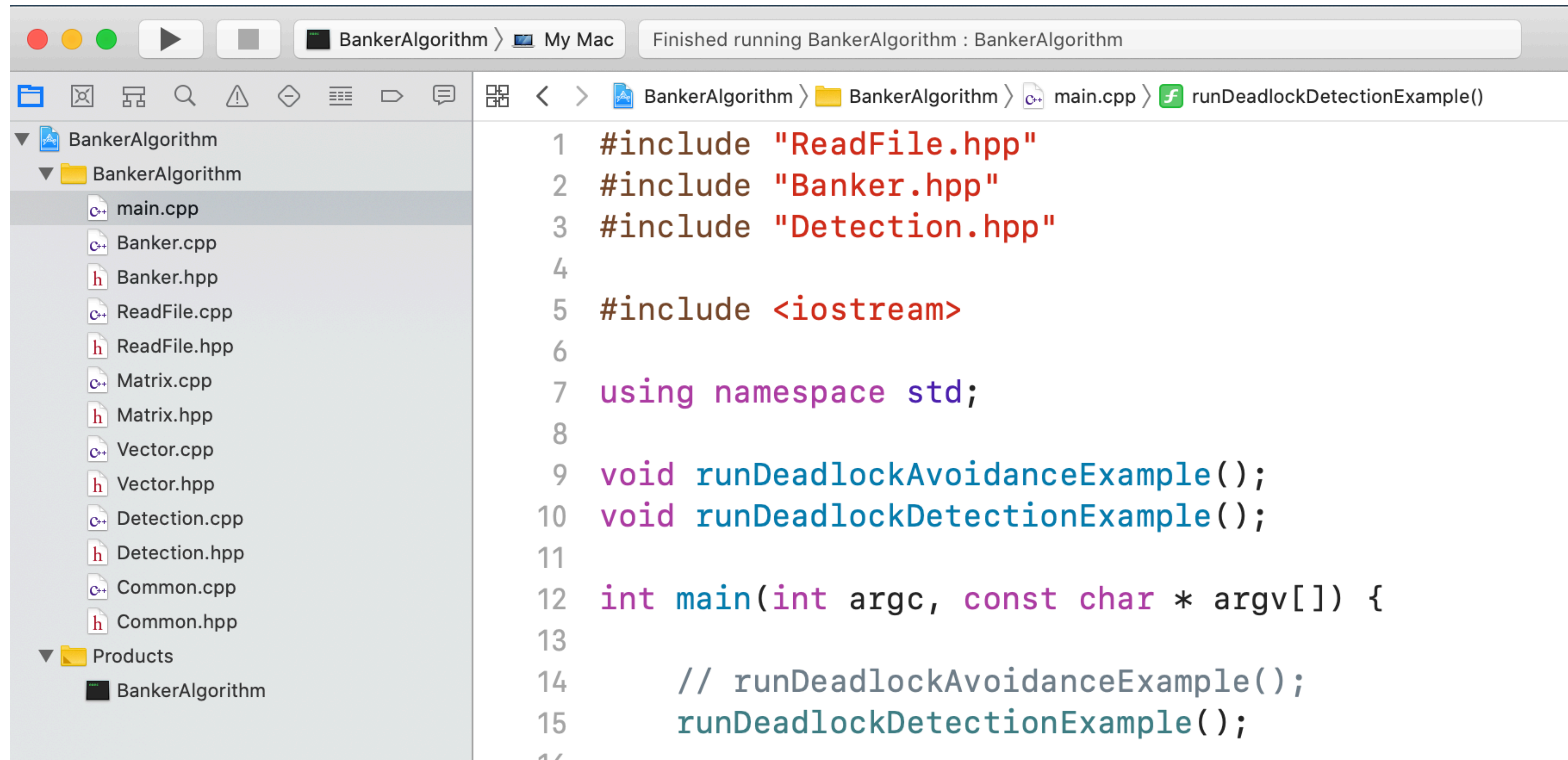
Example

- Five Threads: T_0 through T_4
- Three resource types A, B, and C
 - A: 7 instances
 - B: 2 instances
 - C: 6 instances

Example

	<u>Allocation</u>	<u>Request</u>	<u>Request</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
T_0	0 1 0	0 0 0	0 0 0
T_1	2 0 0	2 0 2	2 0 2
T_2	3 0 3	0 0 0	0 0 1
T_3	2 1 1	1 0 0	1 0 0
T_4	0 0 2	0 0 2	0 0 2

Implementation



```
1 #include "ReadFile.hpp"
2 #include "Banker.hpp"
3 #include "Detection.hpp"
4
5 #include <iostream>
6
7 using namespace std;
8
9 void runDeadlockAvoidanceExample();
10 void runDeadlockDetectionExample();
11
12 int main(int argc, const char * argv[]) {
13
14     // runDeadlockAvoidanceExample();
15     runDeadlockDetectionExample();
16 }
```

Outline

- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock → kill 掉!
→ reset 掉!

Recovery from Deadlock

- There are two options for breaking a deadlock
 - Abort one or more threads to break the circular wait
└ 把一个 / 多个 thread 杀掉
 - Preempt some resources from one or more of the deadlocked threads

Process and Thread Termination

- Two methods
 - Abort all deadlocked processes → 一次就全殺掉
 - Abort one process at a time until the deadlock cycle is eliminated → 一次殺掉一个直到沒有cycle
 - How to choose the process
 - What the priority of the process is
 - How long the process has computed and how much longer the process will compute before completing its designated task
 - How many and what types of resources the process has used
 - How many more resources the process needs
 - How many processes will need to be terminated

Resource Preemption

- Three issues need to be addressed
 - Selecting a victim
 - Rollback
 - We must roll back the process to some safe state and restart it from that state
 - Starvation